Day 100: Named Entity Recognition with a Pretrained Model on a Real Dataset

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 15, Day 3 | Day 100 of 365 | Complete study guide

Watch: Named Entity Recognition - using BERT model (Search Techie) | Named Entity Recognition, Part 5: token classification code (Gyan DEEP)

Script: `day100_named_entity_recognition.py` · every number in this guide is copied by program from `day100_run_output.txt`, the real output of that script.

Day 100 of the roadmap. Named entity recognition labels every word as part of a person, organisation, location or other name, or none. On the genuine held-out test set of CoNLL-2003 (Reuters news), a pretrained DistilBERT gets 0.8854 entity-level F1, scored from scratch. A gazetteer of every name seen in training gets 0.5965, and finds 0.0000 of the names it never saw, where the model finds 0.8612. Per-word accuracy (0.9780) looks better than either, because 0.8253 of words are not names at all.

0. Where today fits

DEFINITION — start here, no prior knowledge assumed

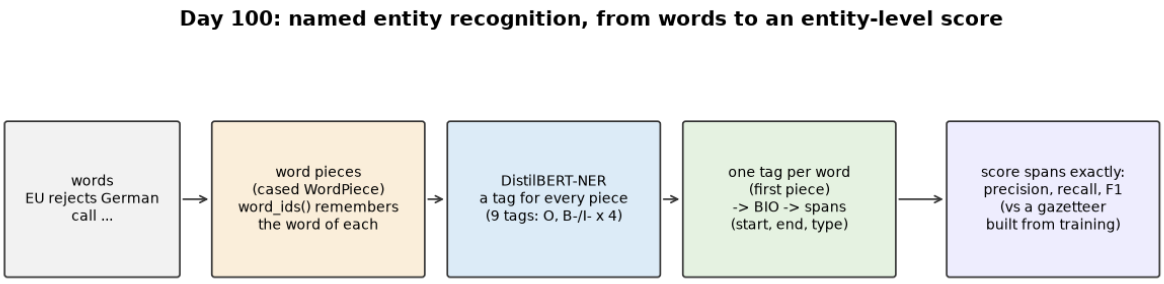
Named entity recognition (NER) gives one label per word. The **BIO** scheme marks the first word of a name B-TYPE, the rest I-TYPE, and every other word O; CoNLL-2003 has four types: PER (people), ORG (organisations), LOC (locations) and MISC (other names: nationalities, events, products).

A **token-classification** model is a Transformer with one small classifier applied at every position, so it outputs a tag for every word piece.

REAL-WORLD SCENARIO

A newsroom wants every article indexed by the people, companies and places it mentions, without anyone reading them all.

SYSTEM DIAGRAM



From words to word pieces, tags, spans and an entity-level score.

Today's part for Day 104

`day100_exercise.py` is the scoring part: `spans` (BIO tags to (start, end, type) spans) and `prf` (exact-match precision, recall, F1), checked on the awkward cases. Its real output:

ACTUAL OUTPUT

```
one entity                    ['B-PER', 'I-PER', 'O'] -> [(0, 2, 'PER')]  correct: True
runs to the end               ['O', 'B-LOC', 'I-LOC'] -> [(1, 3, 'LOC')]  correct: True
two of one type side by side  ['B-PER', 'B-PER', 'I-PER'] -> [(0, 1, 'PER'), (1, 3, 'PER')]
    correct: True
stray I- with no B-           ['O', 'I-ORG', 'I-ORG', 'O'] -> [(1, 3, 'ORG')]  correct: True
type change inside I- run     ['B-ORG', 'I-ORG', 'I-LOC'] -> [(0, 2, 'ORG'), (2, 3, 'LOC')]
    correct: True
nothing                       ['O', 'O'] -> []  correct: True
3 gold spans, 4 predicted, 1 exactly right: precision 0.250 (1/4), recall 0.333 (1/3), F1 0.286
a wrong type or a boundary off by one word earns nothing, exactly as in the CoNLL evaluation
```

Data and model: CoNLL-2003 comes from the Hugging Face hub's parquet copy (the original repository uses a loading script the current library no longer runs); dslim/distilbert-NER was fine-tuned by its author on CoNLL-2003's training set, so the test set used here is genuinely held out.

How to read each section

Step	What it gives you
DEFINITION	What the idea is, in plain words. Start here.
REAL-WORLD SCENARIO	An everyday situation where the same idea shows up.
MATHEMATICS	The formula, when the section introduces one.
IMPLEMENTATION	The real code from the script.
ACTUAL OUTPUT	What the script really printed, copied unchanged.
What it means	How to read the numbers, and what they do and do not show.
Why choose this? / Why NOT this?	When the idea is the right tool, and what it costs.

1. The task and the data

DEFINITION — start here, no prior knowledge assumed

CoNLL-2003's English part: Reuters news from 1996, every word tagged by hand. Training and test come from different days of news, so many test names were never seen in training.

REAL-WORLD SCENARIO

Highlighting names in a newspaper with four colours of marker, one per type.

ACTUAL OUTPUT

```
a training sentence, word by word:
  EU/B-ORG  rejects/O  German/B-MISC  call/O  to/O  boycott/O  British/B-MISC  lamb/O  ./O
train: 14,041 sentences, 203,621 words, 23,499 entities (PER 6,600, ORG 6,321, LOC 7,140, MISC 3,438); words
    tagged O: 0.833
test: 3,453 sentences, 46,435 words, 5,648 entities (PER 1,617, ORG 1,661, LOC 1,668, MISC 702); words tagged
    O: 0.825
Actual output from this lesson's run.
```

What it means

The test set has 5,648 entities in 46,435 words, spread fairly evenly over PER, ORG and LOC with fewer MISC (702). About 0.825 of words are O: most words are not names, which matters for how the task is scored.

WHY CHOOSE THIS?

A long-standing, shared benchmark: results are comparable across papers.

WHY NOT THIS? (TRADEOFFS)

1996 news only; modern names, other genres and other languages are not tested.

2. Words and word pieces

DEFINITION — start here, no prior knowledge assumed

CoNLL gives one tag per *word*, but the model sees *pieces*: 'Ouagadougou' becomes six. `word_ids()` records which word each piece came from, and the common convention (used here) is to read each word's tag from its **first** piece.

REAL-WORLD SCENARIO

Grading a spelling test by the first letter of each word you wrote, because that is where the answer is recorded.

IMPLEMENTATION

```
def predict_tags(tok, model, sents, bs=32):
    out = []
    with torch.no_grad():
        for i in range(0, len(sents), bs):
            batch = sents[i:i + bs]
            enc = tok(batch, is_split_into_words=True, padding=True, truncation=True,
                return_tensors="pt")
            pred = model(**enc).logits.argmax(-1).numpy()
            for b, words in enumerate(batch):
                tags, seen = [], set()
                for pos, w in enumerate(enc.word_ids(b)):
                    if w is not None and w not in seen:
                        tags.append(TAGS[pred[b, pos]]); seen.add(w)
                tags += ["O"] * (len(words) - len(tags))
                out.append(tags)
    return out
```

ACTUAL OUTPUT

```
words: ['Schwarzenegger', 'visited', 'Ouagadougou', 'with', 'Lufthansa', '.']
pieces: ['[CLS]', 'Sc', '##hwa', '##rz', '##ene', '##gger', 'visited', 'O', '##ua', '##gado', '##ug', '##ou',
    'with', 'Lu', '##ft', '##han', '##sa', '.', '[SEP]']
word of each piece: [None, 0, 0, 0, 0, 0, 1, 2, 2, 2, 2, 2, 3, 4, 4, 4, 4, 5, None]
tag of each piece: [CLS]:O Sc:B-PER ##hwa:B-PER ##rz:B-PER ##ene:B-PER ##gger:B-PER visited:O O:B-LOC
    ##ua:B-LOC ##gado:B-LOC ##ug:B-LOC ##ou:B-LOC with:O Lu:B-PER ##ft:B-PER ##han:B-PER ##sa:B-PER .:O
    [SEP]:O
one tag per word (first piece): ['B-PER', 'O', 'B-LOC', 'O', 'B-PER', 'O']
Actual output from this lesson's run.
```

What it means

Rare names are cut into many pieces, and this model tags every piece of a name B-PER (probably because it was trained with the word's tag copied to every piece). Read naively, 'Schwarzenegger' would become five one-piece people; taking the first piece's tag gives one B-PER per word, which is what the scorer expects. The alignment step is small but easy to get wrong.

WHY CHOOSE THIS?

One rule maps the model's pieces back to the dataset's words.

WHY NOT THIS? (TRADEOFFS)

It ignores the other pieces' opinions; voting across pieces is an alternative (not tested).

3. Scoring: entity spans, not words

DEFINITION — start here, no prior knowledge assumed

Tags become **spans** (first word, end, type). A predicted span counts only if its words *and* type exactly match a gold span. Precision = right / predicted, recall = right / gold, F1 = their harmonic mean. This is the CoNLL evaluation's rule.

REAL-WORLD SCENARIO

A treasure hunt where 'next to the right tree' scores nothing: only the exact spot counts.

MATHEMATICS

$$P = |gold \cap pred| / |pred|, \quad R = |gold \cap pred| / |gold|, \quad F1 = 2PR / (P + R), \quad spans\ (start, end, type)\ matched\ exactly$$

IMPLEMENTATION

```
def spans(tags):
    out, start, typ = set(), None, None
    for i, t in enumerate(tags + ["O"]):
        if t == "O" or t.startswith("B-") or (t.startswith("I-") and t[2:] != typ):
            if start is not None:
                out.add((start, i, typ))
            start, typ = (i, t[2:]) if t != "O" else (None, None)
    return out
def prf(gold, pred):
    tp = sum(len(g & p) for g, p in zip(gold, pred))
    n_pred, n_gold = sum(map(len, pred)), sum(map(len, gold))
    p, r = tp / max(1, n_pred), tp / max(1, n_gold)
    return p, r, (2 * p * r / (p + r) if p + r else 0.0), tp, n_pred, n_gold
```

ACTUAL OUTPUT

```
per-word tag accuracy 0.9780 (tagging every word O would already score 0.8253)
entity level: 5,648 gold entities, 5,804 predicted, 5,070 exactly right -> precision 0.8735,
    recall 0.8977, F1 0.8854
type     gold  precision   recall      F1
PER     1,617     0.9419   0.9524  0.9471
ORG     1,661     0.8440   0.8694  0.8565
LOC     1,668     0.9004   0.9209  0.9105
MISC      702     0.7314   0.7835  0.7565
Actual output from this lesson's run.
```

What it means

Per-word accuracy, 0.9780, flatters: tagging every word O would already get 0.8253. The entity-level F1, 0.8854 (5,070 of 5,648 names exactly right), is the honest number. People are easiest (0.9471), MISC hardest (0.7565): 'Japanese', 'World Cup' and 'Windows' are one type with little in common.

WHY CHOOSE THIS?

Strict, standard, and directly comparable with published results.

WHY NOT THIS? (TRADEOFFS)

A boundary off by one word gives no credit at all, even when the name was clearly found.

4. A from-scratch baseline: the gazetteer

DEFINITION — start here, no prior knowledge assumed

A **gazetteer** is a list of names. Built from every entity in CoNLL's training set (with its most common type), matched in test sentences longest-first. No learning beyond memorising.

REAL-WORLD SCENARIO

A receptionist with a list of every known client: fast and exact for regulars, useless for a new face.

IMPLEMENTATION

```
class Gazetteer:
    def __init__(self, sents, tags):
        votes = {}
        for words, t in zip(sents, tags):
            for s, e, typ in sorted(spans(t)):
                votes.setdefault(tuple(words[s:e]), Counter())[typ] += 1
        self.names = {k: c.most_common(1)[0][0] for k, c in votes.items()}
        self.longest = max(map(len, self.names))
    def tag(self, words):
        tags, i = ["O"] * len(words), 0
        while i < len(words):
            for L in range(min(self.longest, len(words) - i), 0, -1):
                typ = self.names.get(tuple(words[i:i + L]))
                if typ:
                    tags[i:i + L] = [f"B-{typ}"] + [f"I-{typ}"] * (L - 1); i += L; break
            else:
                i += 1
        return tags
```

ACTUAL OUTPUT

```
8,082 distinct entity strings in training (longest 10 words), each with its most common type
system                      precision   recall      F1
gazetteer (from scratch)       0.7196   0.5094  0.5965
DistilBERT-NER                 0.8735   0.8977  0.8854
Actual output from this lesson's run.
```

What it means

Memorising 8,082 names gives F1 0.5965: precise enough (0.7196), but it finds only half the names (0.5094). The model, 0.8854, is far ahead.

WHY CHOOSE THIS?

A useful sanity check: any learned model must beat it.

WHY NOT THIS? (TRADEOFFS)

Cannot find a name it has not seen, and ignores context ('Washington' the place or the person).

5. Where the model fails

DEFINITION — start here, no prior knowledge assumed

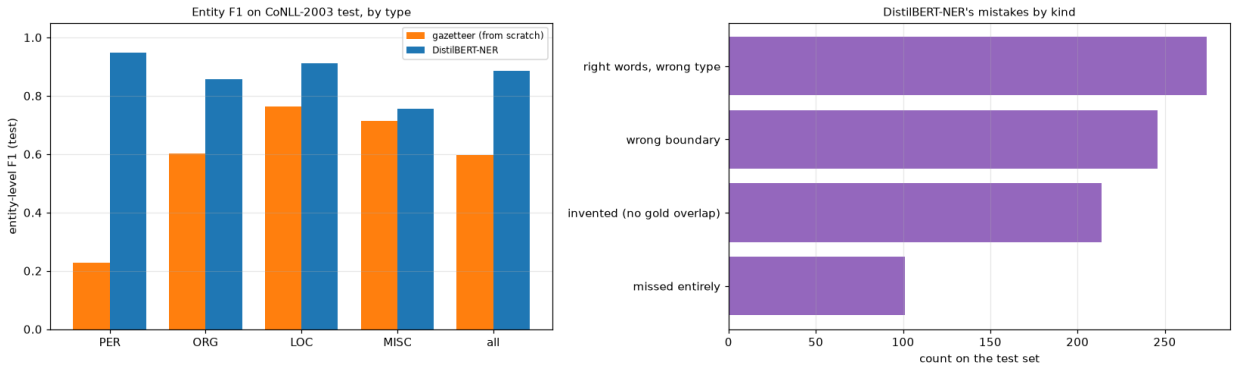
Gold entities split into those whose exact text appeared as an entity in training and those that are new; and every mistake classified: right words but the wrong type, a wrong boundary, a span invented from nothing, or a name missed entirely.

REAL-WORLD SCENARIO

Checking whether a student understood the rule or memorised the examples: give them new examples.

ACTUAL OUTPUT

```
DistilBERT-NER  recall on gold entities new: 0.8612 of 2,600; seen in training: 0.9288 of 3,048
gazetteer       recall on gold entities new: 0.0000 of 2,600; seen in training: 0.9439 of 3,048
the model's mistakes by kind: right words, wrong type 274, wrong boundary 246, invented (no gold overlap) 214,
    missed entirely 101
wrong-type confusions (gold -> predicted), most common: ORG->LOC 49, LOC->ORG 41, ORG->MISC 34, MISC->ORG 34,
    ORG->PER 28
Actual output from this lesson's run.
```



Left: entity F1 by type for the gazetteer and the model. Right: the model's mistakes by kind.

What it means

The gazetteer finds 0.0000 of the 2,600 new names, by construction. The model finds 0.8612 of them: it uses context and the shape of words, not a list. It is still better on names it saw (0.9288). Its mistakes are spread across kinds: 274 right-span-wrong-type (mostly ORG and LOC swapped, 49 and 41 times; a likely cause, not checked here: in CoNLL a city's name used for its sports team is ORG), 246 wrong boundaries, 214 invented and 101 missed.

WHY CHOOSE THIS?

Shows what kind of error to fix: boundaries, types or coverage.

WHY NOT THIS? (TRADEOFFS)

One test set from one year of one news agency.

6. A check against the library's pipeline

DEFINITION — start here, no prior knowledge assumed

The library's token-classification pipeline, with its own word grouping ('first' strategy), against this script's spans on a new sentence.

REAL-WORLD SCENARIO

Checking your own long division against a calculator once.

ACTUAL OUTPUT

```
text: Angela Merkel met Emmanuel Macron in Paris on Tuesday to discuss the European Union budget.
pipeline: Angela Merkel (PER, 0.999); Emmanuel Macron (PER, 0.999); Paris (LOC, 0.998); European
    Union (ORG, 0.985)
ours:     Angela Merkel (PER); Emmanuel Macron (PER); Paris (LOC); European Union (ORG)
Actual output from this lesson's run.
```

What it means

The two agree exactly: the same four entities with the same types. The from-scratch alignment and span code does what the library does.

WHY CHOOSE THIS?

A quick confirmation of the whole decoding path.

WHY NOT THIS? (TRADEOFFS)

One sentence is a spot check, not a proof.

Interview preparation — technical questions

Q: What is the BIO scheme?

A: B- starts a name, I- continues it, O is outside; spans are read off by where B- and type changes occur.

Q: How do you align word pieces with word labels?

A: Use the tokenizer's word_ids(); take each word's first piece's prediction.

Q: Why is per-word accuracy misleading for NER?

A: Most words are O: tagging everything O scores 0.8253 here.

Q: How is entity-level F1 computed?

A: Exact span-and-type matches: precision over predictions, recall over gold, harmonic mean.

Q: What is a gazetteer baseline?

A: A list of known names matched in text; here F1 0.5965, recall 0 on unseen names.

Interview preparation — theoretical questions

Q: Why does MISC score lowest?

A: It mixes unrelated kinds of names, so there is no single pattern to learn.

Q: Why do ORG and LOC get confused?

A: The same word can be both: a city or the team named after it.

Q: How can a model find names it never saw?

A: From context ('said ... chief executive') and word shape (capitals, endings).

Q: What does a stray I- tag mean?

A: A continuation with no beginning; the convention used here starts a new span there.

Key takeaway and what's next

> **KEY TAKEAWAY**
> NER is scored on whole names, exactly. A pretrained token classifier reaches 0.8854 F1 on CoNLL-2003's real test set and finds 0.8612 of names it never saw, where a list of known names finds none. The details that decide the score are mundane: aligning pieces with words and decoding BIO tags correctly.

- Entity F1 0.8854 (P 0.8735, R 0.8977); PER 0.9471, ORG 0.8565, LOC 0.9105, MISC 0.7565.
- Per-word accuracy 0.9780 against 0.8253 for all-O.
- Gazetteer 0.5965; new names: model 0.8612, gazetteer 0.0000.
- Mistakes: 274 wrong type, 246 wrong boundary, 214 invented, 101 missed.

Suggested watch order, if starting from zero

- 1. Named Entity Recognition - using BERT model (Search Techie) — what NER is and how a BERT model does it.
- 2. Named Entity Recognition, Part 5: token classification code (Gyan DEEP) — the token-classification code in practice.

Day 101 preview

- **Summarization.** Extractive (choose sentences, from scratch) against abstractive (a pretrained T5 writes a new summary), on real news, scored with Day 97's ROUGE and checked for invented details.